

DRAFT — Diskless Viewstamped Replication with Unbounded Crash-Stop Reincarnation

Simon Massey

Abstract—Small replicated metadata should not require a disk flush on the critical path. Unbounded Viewstamped Replication Revisited is a protocol design for embedding strong consistency with volatile normal-path replication and crash-stop reincarnation. A cleanly stopped process restarts with its identity and flushed state. A crashed process permanently retires its old identity and rejoins under a fresh identity, obtaining state and voting authority through reconfiguration. Its organising mechanism is leader overlap: where the leader is the sole intersection of the active decision quorum and the next prepare quorum, it withholds its own promise so that preparation can proceed while the active quorum remains usable. Five-voter replacement uses six committed batches, including exact doubling and halving. A constructive Rust solver also computes paths between arbitrary weighted-majority configurations with an available majority at each endpoint. The Lean development proves weighted reachability and a phantom-identity casting-vote construction, alongside agreement and identity exclusion for a three-node schedule and its five-voter extension. The addendum published with the code separates the proof contracts and executable checks.

Index Terms—Crash-stop, leader overlap, reconfiguration, strong consistency, Viewstamped Replication.

I. INTRODUCTION

Oki and Liskov introduced Viewstamped Replication in 1988 [1]. Liskov and Cowling’s 2012 Viewstamped Replication Revisited (VRR) eliminated disk writes during normal processing and view changes [2]. Replication therefore retained the state needed for agreement without adding a local disk flush to each update. The remaining difficulty was how to bring a replica back after loss of that state.

Michael *et al.* exposed a safety failure in VRR’s recovery mechanism in 2016 [3, Sec. 4.2 and Appendix A.1]. A replica can send a view-change commitment, crash, and return in an older view having forgotten the commitment. It can then help commit an operation that a later leader loses when delayed view-change messages arrive. Their counterexample breaks linearizability, including when channels preserve message order. The authors provide a virtual stable storage construction to support diskless crash recovery. Their result makes the problem precise: restoring a replica’s data must also preserve the commitments made by its identity.

Unbounded Viewstamped Replication Revisited (uVRR) repairs this failure by making a crash final for the protocol identity. The crashed identity never resumes execution. The process that replaces it joins under a fresh identity, receives state from the cluster, and acquires voting authority through committed membership changes. Messages sent before the crash remain messages of the old identity; the new process

cannot add fresh votes to that identity’s history. A cleanly stopped process can instead restart under its existing identity because it has preserved its protocol state. Startup fencing and clean shutdown use durable writes, keeping those writes outside normal replication. Loss of volatile state therefore leads to replacement of a member: crash recovery becomes crash-stop reincarnation.

Turner supplies the construction that connects this lifecycle to continued replication [9]. Where the leader is the sole intersection of an active decision quorum and the next prepare quorum, it can withhold its own promise while collecting the others. The active quorum remains usable during that preparation; the leader completes the handover with its own local promise. We use this leader overlap to organise reincarnation. For three unit-weight voters, replacement takes two committed batches. For five, the weighted schedule has six, including exact doubling and halving. The addendum published with the code [24] gives the proof ladder, a general configuration-path constructor and an abstract casting-vote completion using fresh identities. Turner’s agreement argument supplies the basis for composing safe adjacent configuration boundaries.

Together, these ideas retain VRR’s diskless normal path while giving failed members a route back through fresh identities. The intended setting is agreement over small amounts of metadata embedded in a distributed application [14]. Such a service can coordinate membership or advisory-lock metadata without putting synchronous persistence on every update’s critical path. Its normal replication need not wait for a disk flush or batch updates to amortise one. The design objective is strong consistency deployable on commodity cloud infrastructure: stock virtual machines, stock load balancing, and no custom networking hardware.

Section II develops the lifecycle and leader-overlap construction. Section III starts with three-voter Crash-Stop-Reincarnation and extends the argument to five voters, stating the history invariants needed for agreement, exclusion of the evicted voter and identity nonreuse. Section V states the machinery that executes such a schedule as one fused message round: the solver that computes and certifies it, and the repetitive construct that telescopes its verification onto the head and tail commands of the packed batch. Section VIII develops the metadata applications; the accompanying addendum gives the mathematical working and verification evidence. The name “unbounded” follows Turner’s treatment of pipelining through reconfiguration: agreement does not depend on a fixed bound on outstanding slots.

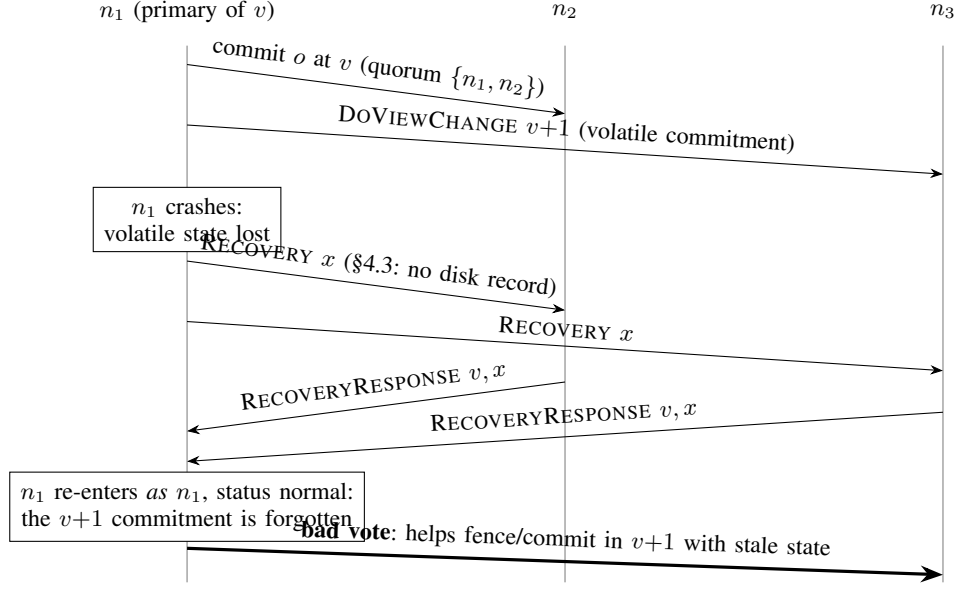


Fig. 1. The diskless-recovery amnesia of VRR, after Michael *et al.* [3, Sec. 4.2 and Appendix A.1]. A replica sends a view-change commitment, crashes, and returns in an older view having forgotten the commitment; it can then help commit an operation that a later leader loses when delayed view-change messages arrive. The counterexample breaks linearizability, including when channels preserve message order. Three voters, $f=1$; acceptance replies and unrelated traffic omitted.

II. METHOD

A. Processes, state and identity

The agreement model is asynchronous and non-Byzantine. Messages may be delayed, and an old message retains the identity that issued it. Each incarnation has a logical identity distinct from its physical host. The lifecycle must ensure that an identity never produces new protocol actions after a replacement incarnation takes over.

A node identity is universally unique and never recycled, and it is durable before any startup emission. This is the obligation Paxos places on proposal numbers [5], applied to node identity: a bump assigns a value that no life of any node has used or will use. The boot marker carries the identity as the durable pair {system identifier, counter}, the packing of a unique per-processor identifier with an increasing counter that Disk Paxos describes for ballot numbers [6]. The identity write is amortised into the single small boot-fence write of the following paragraph, so no flush is added to any critical path.

A clean stop first quiesces protocol processing, then flushes the state required for restart, and finally records a durable stopped marker. On startup, a valid stopped marker permits the process to reload that state and enter *Restarting* with the same identity. Before sending protocol messages it durably records that it is running again. This boot fence prevents a later crash from being mistaken for another clean restart. A restarting process retains its promises and accepted state, so ordinary timeout processing is permitted by the design.

Without a valid clean-stop marker, startup enters *Joining* under a fresh identity. The old identity is crash-stopped. Cached state may reduce transfer work, but it provides no voting authority. The joiner asks the leader to evict the old incarnation and admit the new one; the leader can send missing state while coordinating the change. A joiner does not vote or

initiate a view change. Its authority comes from the committed configuration, never from the fact that it shares a host with a former voter.

The implementation design uses replicated superblock markers and majority interpretation of valid marker copies. This paper abstracts that mechanism as correct clean-stop classification and durable identity fencing. The Lean result concerns the resulting lifecycle; it does not model the storage protocol itself.

B. Quorums across eras

An era determines the quorum families for a suffix of log slots. Write Q_e^I for its phase-one family and Q_e^{II} for its phase-two family. The history model requires the following within-era and adjacent-era intersections:

$$q \cap r \neq \emptyset \quad (q \in Q_e^{II}, r \in Q_e^I), \quad (1)$$

$$q \cap r \neq \emptyset \quad (q \in Q_e^I, r \in Q_{e+1}^{II}). \quad (2)$$

The direction in (2) matters: the earlier phase-one quorum must meet the later decision quorum. These conditions are part of Turner's era-based agreement argument [9].

Our concrete schedule uses the same strict weighted-majority family for both phases. If $w_e(n)$ is a nonnegative integer weight on a finite support, a quorum q satisfies

$$2 \sum_{n \in q} w_e(n) > \sum_n w_e(n). \quad (3)$$

The weighted proof establishes the requisite intersections when successive configurations differ by at most one unit of total absolute weight. Membership records for zero-weight nodes may change in the same batch without altering (3). A crash itself does not change any weight; only committed configuration commands do.

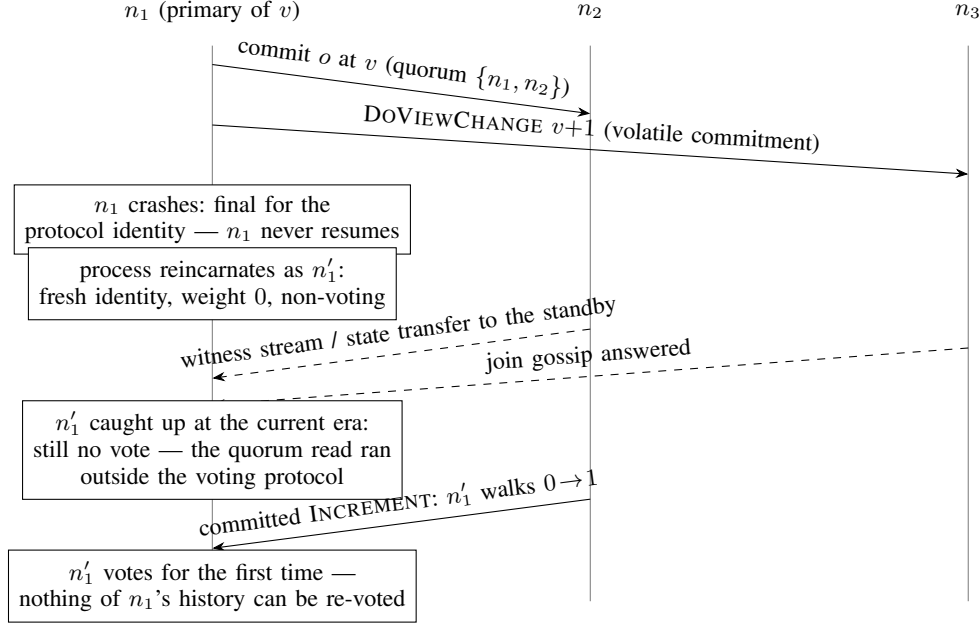


Fig. 2. Crash-stop reincarnation on the same prefix. The crashed identity never resumes; the replacement joins as a weight-0 standby, and its quorum read and state transfer complete while it cannot vote, so the amnesia’s bad vote is unrepresentable rather than filtered. Promotion to voting weight is itself a committed membership change. Time runs downwards without a scale; acceptance replies omitted.

III. THREE-NODE CRASH-STOP-REINCARNATION

Suppose a leader ℓ uses ballot b with an active decision quorum q . To prepare ballot $b' > b$ for the next era, it chooses a prospective phase-one quorum q' with

$$q \in \mathcal{Q}_e^{\Pi}, \quad q' \in \mathcal{Q}_{e+1}^{\text{I}}, \quad q \cap q' = \{\ell\}. \quad (4)$$

This is Turner’s casting-vote condition [9]. The leader first sends prepare requests only to $q' \setminus \{\ell\}$. Their promises do not disable an acceptor in q . The leader withholds its own promise and may continue proposing at b , provided no other event invalidates the old ballot’s guards.

After receiving those promises, the leader chooses the suffix boundary k and makes its own promise locally. Phase one is then complete without another network exchange for the leader’s response. Proposals at b' must still obey the normal rule for preserving values reported by phase one. Figure 3 shows the message order.

Decision quorums follow the era of the *slot*, not simply the era associated with a ballot. Consequently, continuing to decide later-era slots at b also requires a quorum valid for those slots. A value accepted under a leader that has failed is committed at the same slot by the new leader’s per-slot install; it is not a new value at the slot and not a violation. In the concrete pivot below, the active quorum is a majority in both adjoining eras.

Table I is the three-node schedule: n_0 fails and is replaced by its fresh identity n_3 , while n_1, n_2 survive. A join inserts a zero-weight identity; an increment grants its voting weight. A leave removes the old identity after its weight reaches zero.

Each boundary preserves cross-era majority intersection by a unit change or exact scaling. Figure 3 illustrates the promotion boundary.

The leader stays live for client requests and drives a forced weight sequence from the voting-weights rules [21]: the exiting node’s weight goes $1 \rightarrow 0$ (a Decrement; the Leave follows at weight zero), the new identity joins at weight 0 (a Join), and the new identity goes $0 \rightarrow 1$. Each step is a unit-weight change, so the strict majorities of consecutive configurations overlap by the unit-change overlap argument and every intermediate era is quorum-safe on its own; a leader dying mid-sequence leaves only safe eras that a new leader can continue. Table I instantiates the sequence at unit weight for the three-voter cluster A, B, C with B the leader and C the crashed identity that returns as D : the start configuration is $(1, 1, 1, \text{~})$; era 1 drives the leader’s batch DECREMENT (C) + JOIN (D), reaching $(1, 1, 0, 0)$; era 2 drives INCREMENT (D) + LEAVE (C), reaching $(1, 1, \text{~}, 1)$. In the table a – is not a weight: it marks an identity that is not a member of the configuration in that era, whereas 0 is the weight of a standby member that the cluster still holds in its order — which the reincarnated node is between the two eras while it is streamed preemptively. The old identity C is evicted by reincarnation, never a recovered amnesiac. If the sequence runs in the doubled scale of a weight-doubling plan, the return halve is integral only when all weights are even, so the corner sequence inserts one extra unit increment on the new identity before the halve. The leader streams the first reconfiguration, then client traffic, and preemptively streams to the reincarnated node as a weight-0 standby; messages to it are fine and messages from it are discarded by the standard membership check. If the leader crashes, the cluster must reach a stable leader before the reincarnated node forces its old identity’s eviction.

Fig. 4 shows this forced sequence as a space-time chart drawn in the grammar of Turner’s message diagrams [9]:

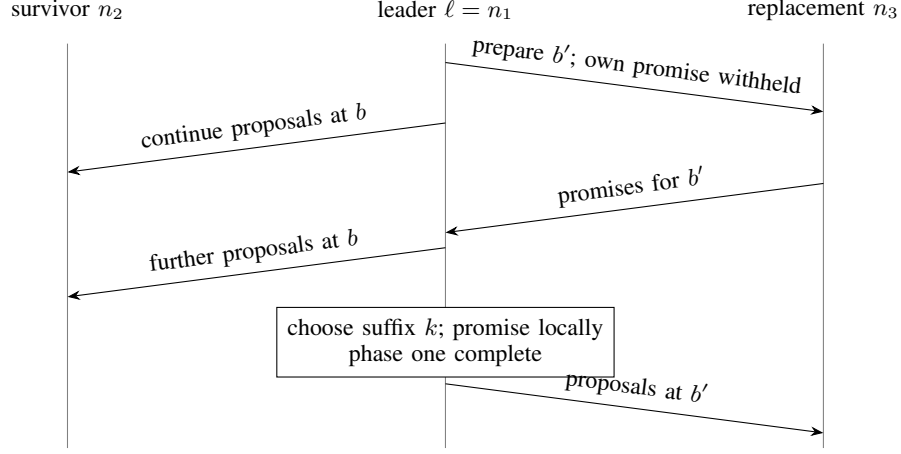


Fig. 3. Three-voter leader overlap at $E_1 \rightarrow E_2$: $q = \{n_1, n_2\}$ and $q' = \{n_1, n_3\}$. Each side outside the leader is a one-node minority. Original schematic of Turner's casting-vote construction [9]. Time runs downwards without a scale; acceptance replies, state transfer and unrelated traffic are omitted.

TABLE I

REINCARNATION FORCED WEIGHT SEQUENCE, THE LEADER B HELD STABLE THROUGHOUT. THE LEADER-VIEW COLUMN RECORDS THE SERVING VIEW AT EACH ERA, COUNTED FROM VIEW 1, THE LEAST VIEW ELECTING B UNDER THE OPENING ARITHMETIC. THE PARENTHESED INCREMENT AT A BOUNDARY IS THE NOMINATION RIDER'S OFFSET: THE LEAST POSITIVE OFFSET RE-ELECTING THE LEADER UNDER THE NEW ARITHMETIC (SECTION V).

Era	A	B	C	D	leader vie
start	1	1	1	—	1
era 1: DECREMENT (C) + JOIN (D)	1	1	0	0	3 (+2)
era 2: INCREMENT (D) + LEAVE (C)	1	1	—	1	4 (+1)

four lifelines A , B (the leader), C , and D , time flowing down; a numbered diagonal vector per step; the bold cross where C crashes and is fenced, with its dead segment dashed below; the open circle where D 's lifeline begins; the dashed era-1 separator carrying DECREMENT (C) + JOIN (D), with D joining at weight 0 as a standby and receiving the dashed preemptive stream; the era-2 separator carrying INCREMENT (D) + LEAVE (C), with D promoted to weight 1 and C leaving at weight 0; and the forming quorum $\{A, B\}$ promised in both eras, so no casting-vote pivot is exercised on this path. The chart is rendered from the tracked source `reincarnation.svg`.

The addendum published with the code [24] provides a Lean theorem that composes an era-based agreement theorem with replacement weights and an abstract identity lifecycle. The work builds upon Turner's P2–P7 [9]: promises preserve their fences, reports identify prior acceptances, proposals preserve the selected history, and chosen values have quorum evidence. A well-founded total ballot order and the ballot/slot era conditions complete the contract. The addendum states each hypothesis and the exercise that connects operational executions to it.

Theorem 1 (Three-voter Crash-Stop-Reincarnation). *For a history using Table I, under the stated order, era and history conditions, any two values chosen for the same slot are equal.*

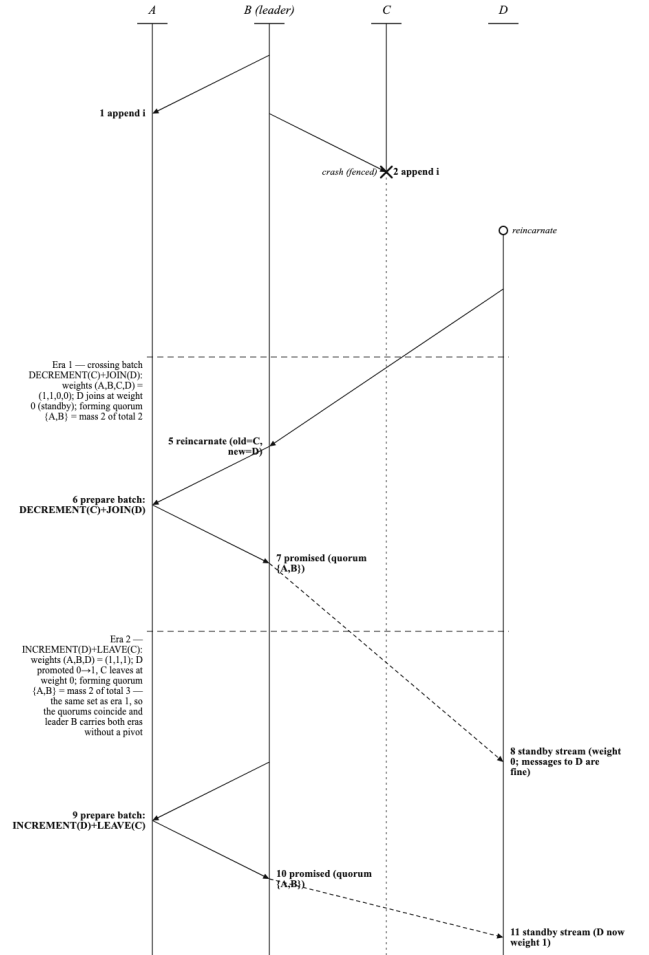


Fig. 4. The reincarnation forced sequence.

The first batch reduces the failed identity to zero voting weight and admits the fresh identity at zero; the second grants the fresh identity unit weight. The retired identity remains without voting authority when subsequent configurations preserve its

TABLE II

FIVE-NODE CRASH-STOP-REINCARNATION: SIX COMMITTED BATCHES, THE LEADER n_1 HELD STABLE THROUGHOUT. ZERO DENOTES VOTING WEIGHT, INCLUDING IDENTITIES ABSENT FROM MEMBERSHIP. THE LEADER-VIEW COLUMN RECORDS THE SERVING VIEW AT EACH STAGE, COUNTED FROM VIEW 1, THE LEAST VIEW ELECTING n_1 UNDER THE OPENING ARITHMETIC. THE PARENTHESED INCREMENT AT A BOUNDARY IS THE NOMINATION RIDER'S OFFSET AT A MOVING BOUNDARY, AND THE FULL VOTER CYCLE OF THE LEADER-PRESERVING VIEW CHANGE AT A SCALING BOUNDARY (DOUBLE, HALVE) (SECTION V).

Stage	n_0	n_1	n_2	n_3	n_4	n_5	leader view
Initial	1	1	1	1	1	0	1
Double	2	2	2	2	2	0	6 (+5)
Join, increment	2	2	2	2	2	1	7 (+1)
Decrement old	1	2	2	2	2	1	13 (+6)
Decrement, leave	0	2	2	2	2	1	15 (+2)
Increment new	0	2	2	2	2	2	20 (+5)
Halve	0	1	1	1	1	1	25 (+5)

exclusion. Along the stipulated identity lifecycle, a superseded identity never becomes current again. In a unit-weight three-voter configuration, any leader has a casting vote between two equal one-node minorities; at the promotion boundary, the surviving leader is the singleton overlap shown in Figure 3.

Proof. The totals in Table I are 3, 2, 3, so the majority thresholds are 2, 2, 2. Each boundary changes one weight by one; the integral weighted-overlap argument gives the within-era and adjacent-era intersections. Turner's agreement theorem applies to those intersections and P2–P7. The two commands establish the stated weights, and identity fencing supplies nonreuse. At the promotion boundary, $\{n_1, n_2\}$ and $\{n_1, n_3\}$ meet only at leader n_1 ; their nonleader members form the two minorities. The leader can collect the replacement's promise while withholding its own. The addendum gives the complete three-node working, including the abstract witness at the first boundary and the state-acquisition obligation. $\square$

IV. FIVE-NODE CRASH-STOP-REINCARNATION

For five original voters, n_0 is replaced by n_5 while $n_1, \dots, n_4$ survive. Table II gives the complete six-batch weighted schedule, including exact doubling and halving.

Both schedules are exercised by the Rust tests, and the addendum [24] gives the arithmetic for both schedules and the general construction of available paths and abstract leader overlaps.

With all five unit voters responding, a leader has the casting pair $\{\ell, a, b\}$ and $\{\ell, c, d\}$. After one voter fails, obtaining the corresponding responding split requires more care. The mechanical transformations in Table II organise the weighted replacement while preserving the agreement invariant; the addendum supplies their safety argument and the distinction between abstract witnesses and protocol receipts.

Theorem 2 (Five-voter Crash-Stop-Reincarnation). *For a history using Table II, under the same order, era and P2–P7 conditions, any two values chosen for the same slot are equal. After the fourth batch the old identity has zero weight; after the sixth the replacement and all four survivors have*

unit weight. Preserving exclusion in subsequent configurations and enforcing the identity lifecycle retain the old identity's retirement.

Proof. Doubling and exact halving preserve the majority family. Each of the four intervening boundaries changes one unit of voting mass. The same local intersection argument therefore applies six times. Induction over these boundaries establishes the era intersection invariant, and Turner's theorem gives agreement. Evaluation of the commands gives the stated final weights. $\square$

V. RECONFIGURATION AS A REPETITIVE CONSTRUCT

The replacement schedules of Sections III and IV are stated longhand: one committed batch per era, the leader streaming the reincarnated standby between batches. This section states the machinery that executes such a schedule as one fused message round: the solver that computes and certifies the schedule before anything is sent, the wire encoding that packs it, and the repetitive construct that reduces its verification to the inspection of the head and the tail of the packed batch. The formalism is the present work's. The quorum geometry and the era-based agreement argument it preserves are Turner's [9]; the underlying consensus invariants are those of Lamport [5].

A. The repetitive construct

The repetitive construct — the name and the guarded notation are due to Dijkstra [26], [27] — is a pattern of assertions over a repetition $\text{do } B \rightarrow S \text{ od}$:

$$\{P\} \quad \text{established before entry,} \quad (5)$$

$$\{P \wedge B\} S \{P\} \quad \text{the body, the induction step,} \quad (6)$$

$$\{P \wedge \neg B\} \quad \text{at the exit.} \quad (7)$$

P is the invariant, B the guard and S the statement: the same statement carries P from each iteration into the next, which is what makes the repetition one induction rather than a sequence of unrelated steps. The pattern is the Rule of Iteration of Hoare [28], D3 of the axiomatic basis: from $\vdash P \wedge B \{S\} P$ conclude $\vdash P \{\text{while } B \text{ do } S\} \neg B \wedge P$. The discipline is that of the 1979 lecture: correctness is a mathematical concern, and one precise argument settles an unbounded number of cases [26]. Two obligations stand apart in the pattern. Preservation is the body's: where B holds and S completes, P holds again. Termination is what supplies $\neg B$ at the exit, and it is a separate proof: an invariant that holds at every step is consistent with a repetition that never ends, so the termination argument stands apart from the preservation argument [27]. Here termination is the finite count the envelope carries.

B. The mapping: one message step per iteration

A node processes a message through one partial function

$$\text{message} \times \text{state} \rightarrow \text{state}' \times \text{list}(\text{messages}),$$

decomposed as one clause per (node role, message type) pair, with guards on the invariants and an exhaustive

match; the sans-IO core of Section IX is this function, made total by its host loop. The fuse is a wire encoding of a schedule over that function, not a new protocol type: $\text{fuse}(c_1, c_2, c_3, \dots)$ hydrates logically to the phase-2 sequence $\text{phase-2}(c_1), \text{phase-2}(c_2), \text{phase-2}(c_3), \dots$ at consecutive slots, the envelope carrying the shared ballot once and the first slot once. Read against the construct, the three names instantiate as follows.

- S is one message step: the application of the next fused phase two. Each application folds one command through the ordinary accept path at its own slot, exactly as though the commands had arrived as separate datagrams with nothing between them: the per-slot guards — the next slot is consecutive, the command is legal in the configuration the fold has established, the promise still permits the ballot, the slot’s era permits the proposal — are evaluated against the in-memory state the preceding applications produced.
- B is the loop over the packed slots: the iteration continues while commands of the envelope remain.
- P is the strong-consistency invariant set: the within-era and adjacent-era intersections (1)–(2), the promise fence of the ballot and the commit frontier’s advance, together with the two operational facts the solver is constructed to preserve at every slot of the schedule — that the leader holds, and that the overlap holds.

The entry assertion (5) is the head guard. The envelope arrives whole or not at all — one datagram of cache-line scale under the transport’s checksum — so the first command decides the whole batch: when $\text{head.ballot} < \text{state.promise}$ or $\text{head.slot} \leq \text{state.commit_frontier}$, the whole envelope is refused, the loop never runs, and there is no partial acceptance and no negative acknowledgement on the wire. The exit assertion (7) is the tail adoption: on completion the node adopts the tail’s ballot as its promise,

$\text{state.promise} \leftarrow \text{tail.ballot},$

one fence covering every packed slot, exactly as if a prepare at that ballot had arrived first: per Lamport, a promise at a ballot covers every lower ballot [5].

C. Telescoping to the head and the tail

Read against the construct, verifying a fused reconfiguration reduces to its ends: the head admits or refuses the envelope whole, and the tail fixes the exit assertion, while the body (6) is the per-slot fold. The mechanised ladder carries the three pieces of this reading, each with its stated scope [24]. The accept direction is finite response-quorum telescoping: a fixed response set eligible at one era is eligible at the next under the preservation hypothesis, and with every responder acknowledging all the packed slots, every slot holds quorum-backed evidence; the concrete eligibility of every row of both replacement schedules — the three rows of the three-voter schedule and the seven rows of the five-voter one — is mechanised beside it. The first transition telescopes the rest: for the three-node two-transition schedule, a base quorum at the first era and a preservation step across the first boundary

give quorum eligibility at every slot through the last era, and the equal-total swap $(1, 2, 1, 2) \rightarrow (2, 1, 2, 1)$ is the negative control — the response set $\{B, D\}$ is a majority at the first era and a minority at the second, so the preservation hypothesis is load-bearing rather than decorative. That theorem assumes the quorum family is preserved across the boundaries; it does not perform phase one, and it is not a refinement of the executable system. The commit direction runs from the tail: the accept path admits an entry only as the next slot, so per-node acceptance is downward closed, and under a fixed configuration — the stable-leader setting of no view change and no era change inside the range, one quorum family, the preservation step the identity — a quorum’s acceptance of the range’s last slot is already its acceptance of every slot in the range, so a commit naming the last slot decides them all; the next-slot guard is load-bearing, since a responder that accepted slot 1 while skipping slot 0 defeats the argument even with the last slot unanimous. Phase-one authority for every packed slot is the header ballot itself: the distinguished leader executes phase one once for all future instances at one proposal number until interrupted [5]. The telescoping records that induction at the envelope’s granularity — one header ballot that is both phase one and phase two for every packed command, the batch processed as a slab whose only interruption point is the envelope boundary — and the quorum telescoping alone performs no phase one.

D. The solver: a runtime proof assistant

The schedule is computed, not configured. A constructive solver — Rust code embedded in the engine, as in the demonstration of Section IX — answers the demand: get from configuration A to configuration B without a violation of the rules. Given the serving configuration and the target, concretely the crashed identity evicted and the fresh identity seated, the solver emits the series of operations that takes one into the other: joins and leaves at weight zero, unit increments and decrements, and exact doubling and halving [21]. Each emitted batch changes at most one unit of total absolute weight or scales every weight exactly, so every intermediate era satisfies the intersections (1)–(2) on its own (Section II). For a response set R that is a strict majority at both endpoints, writing $M_w(R) = \sum_{n \in R} w(n)$ and $T(w) = \sum_n w(n)$, the construction keeps the margin

$$\mu_w(R) = 2M_w(R) - T(w) > 0$$

at every row: the edits move only towards their targets, so the margin never falls below the strictly positive final margin, each unit edit preserves the cross-era intersection, exact scaling preserves the quorum families, and zero-weight membership edits touch neither. The plan the solver emits is its own certificate, the invariants verified against every boundary before a single message is sent: a formal proof assistant embedded in the running system, and no prior system does this. Plans arrive on the leader’s dedicated admin ingress, which the host polls before the regular client queue at every selection, so a reincarnation request never waits behind client traffic. A step the gates refuse, because the cluster changed

underneath the plan, aborts the execution and the operator re-plans from the configuration that committed; a leader dying mid-sequence leaves only safe eras that a new leader can continue (Section III).

E. Holding the leader: the view increment

A ballot is the pair $(era, view)$ under the lexicographic order; a host may carry it as one scalar wherever the encoding is strictly monotone into a well-founded strict total order, and the agreement proof does not know which representation the wire carried [24]. Under the view’s era configuration the leader is a ballot outcome, not a function of membership:

$$\ell(v) = voters[v \bmod |voters|],$$

the positive-weight sequence in membership order. A boundary that changes that sequence moves the arithmetic mid-plan: at an unmoved view number the new arithmetic may name a different node, and no node can independently compute a stable leader through the transition. The nomination rider settles the assignment at the committed slot itself. The command `Nominate{from, u}` rides as the last sub-operation of every establishing batch that carries no scaling operation — a solitary nomination is refused, and a zero offset is refused — and a node applies it as a compare-and-swap: serving the view *from*, it publishes `Ballot{era', from + u}`, the era component being the era the batch’s own fold established, so at a wrap the bump is the era-boundary crossing and no view-change message is owed there. The emission names the least positive offset re-electing the serving leader,

$$u = \min\{u' > 0 : \ell_{next}(v + u') = \ell_{prev}(v)\}.$$

The rider shares the wrap’s establishing slot, so there is no era in which the modulo rule names the wrong leader and none in which the bump has not yet landed. A new leader committing an old leader’s nomination finds the compare-and-swap stale and commits it as a no-op. A step that evicts the serving leader cannot re-elect it, and names the era-entering increment alone. The jump’s safety licence is the view-jump lemma: a view may jump by more than one during a reconfiguration, because no hypothesis of the agreement argument quantifies over intermediate ballots; eras may not jump, the adjacency discipline of the era map [24]. Scaling boundaries are the remaining case: doubling and halving preserve every entry of the positive-weight sequence, so the leader never moves at one, and the era crossing is the leader-preserving view change to the least view past the running one that selects the same leader. Tables I and II record the increment at each boundary of the two replacement schedules, the leader held stable throughout — B for the three-voter schedule, n_1 for the five-voter one, each counted from serving view 1, the least view electing the leader under the opening arithmetic. At a moving boundary the parenthesised figure is the rider’s offset; at a scaling boundary it is the full voter cycle of the leader-preserving view change.

F. The reincarnation budget

The packing and the construct together remove the per-message round trips. The solver computes the whole schedule

before anything is emitted; the leader packs it as one envelope per recipient; one request–reply round collects the fused acknowledgements; one batch-commit broadcast follows, and the reincarnated node learns of its membership on the push of the telescoped reconfiguration. The three-voter replacement carries no scaling operation, so both of its boundaries cross on their riders inside the same commits: the whole reincarnation completes in one round trip. The five-voter schedule packs the same way, its two scaling boundaries crossed by the leader-preserving view change between the envelopes of its unit-mass batches. End to end the rejoin is the classic two round trips, the theoretical Paxos optimum [5]: the gossip answer of Section VI and the reconfiguration round. No forced flush stands anywhere on the path — the boot fence is written only at a clean shutdown and asynchronously at startup, so neither reconfiguration nor reincarnation pays a disk flush (Section II) — the deferred-flush economics that make the round affordable at all [20]. A crashed node therefore reads only a trivial amount of state at startup: the demonstration process boots in double-digit milliseconds including the fence reads, the node rejoins in two round trips, and the cluster returns to full capacity in dozens of milliseconds on commodity hardware.

VI. REJOIN GOSSIP AND WITNESSES

The replacement schedules of Sections III and IV return a failed member to voting weight through committed batches. They presume the replacement can reach a fence: it must know an era the cluster has committed before its engine sees a message. A node whose durable state is lost returns blank; a node that reincarnates under a fresh identity holds no roster entry at all. Neither can guess an era, and the one-era window drops what it cannot span. The rejoin path closes this with a gossip layer outside the voting protocol; the addendum [24] gives its full construction, its worked scenarios and the kernel-checked safety conditions. Three properties carry the design, none of which touches a quorum rule.

a) *The join gossip*: A node outside the cluster never assumes the cluster will come to it. It broadcasts a join request to every node it knows, carrying its frontiers, under the deployment’s authenticated transport. Any node that hears the request answers with the (era, membership) pair it has committed locally; era advances only when a committed batch is applied, so no honest response names an era the cluster has not committed. The joiner adopts the maximum reported era with that response’s membership, flushes the pair to durable storage whenever the adoption changes, and keeps its own retransmission loop: a request lost in flight is re-sent naming the current frontiers, and the resend loop is the gossip’s own reliability rather than a separate discovery path. The joiner never assumes it has heard the latest era, so it always sends to all nodes it knows.

b) *The gossip-witness list*: Every node that hears a join request records the sender on an operator visible witness list; the leader alone acts on the list. It replays the contiguous suffix of its journal from the advertised frontier, in order, each entry through the ordinary accept path — there is no witness-specific accept path — and then streams every subsequent phase-2

and commit to the witness. The witness never votes: quorum membership is decided by the configuration alone. Because the list rides at every node, a successor leader resumes the stream on election without waiting for a re-announcement, so failover does not interrupt keep-up. When a committed batch makes the witness a member, every node removes the entry; delivery as both voter and witness in the window between the commit and the removal is covered by deduplication and merely wastes IO. A node registered as a witness in the startup configuration is never purged: it is an append-only audit tape of everything the cluster committed, and a disaster-recovery sink in another region costs exactly the stream that reincarnation already needs, under every leader in turn.

c) Why the stream is safe: The stream telescopes a promise: per Lamport, a promise at ballot b covers every lower ballot, so a phase-2 accepted at view v raises the witness's promise floor to v exactly as if a prepare at v had arrived first. The witness forfeits nothing by raising the floor — its promises are counted in no quorum — and the leader's right to send at v was earned from members by the ordinary phase one. The addendum states this as five construction rules, H1–H5, and proves the consequence in Lean: the adopted era never exceeds the committed era; once the leader answers, it equals it; a journal extended by the in-order contiguous suffix reconstructs the leader's prefix; the promoted witness's committed prefix is indistinguishable from a member that caught up by ordinary state transfer; and the telescoped promise is itself an equivalence over the witness's accept history — promised at the induced floor, the witness forbids every later acceptance below it, carries the promise state of a member that processed the phase-1 at that view, and promotes with its floor equal to the leader's view. A node that rejoins through this path arrives at its first fence at the cluster's current era, having seen a commit, which makes the one-era window gate unreachable by construction rather than widened.

VII. EVIDENCE

The addendum [24] maps these arguments to Lean 4 [25], bounded exhaustive enumeration, Rust property tests and protocol tests. It includes the existing specialised machine theorem's precise scope, the general lemmas used above, and the reproduction commands. Configuration reachability, abstract casting witnesses and the operational history contract are stated separately so each result can be checked at its own layer.

VIII. DISCUSSION

A. Deployment without custom ordering hardware

The latency literature contains two shorter client paths than the one taken here, and both buy their round trips with infrastructure this design deliberately does not require. NOPaxos orders requests in the network itself: a sequencer stamps a global sequence onto packets in flight, so replicas process a common order and ordinary operations complete in about one client round trip [10]. The evaluated fast path uses programmable switching hardware or an SDN control plane; the ordering point is distinguished infrastructure. CURP lets

a client complete in one round trip by writing concurrently to the master, which orders and executes, and to witnesses, which retain the request unordered [11]. The fast path is sound only while every outstanding operation whose order might be lost is mutually commutative; on a conflict the protocol returns to an ordered replication sync, and the client must reach the master and the witnesses directly rather than any member of the cluster. CORFU shows that a global sequencer need not be custom hardware: its token sequencer is user-space software, with sequencer identity carried in the replicated projection and replaced through reconfiguration [15]. Its clients still obtain a log position and then write to it, two causally dependent stages.

This design takes a different trade. A client reaches the service through an ordinary load balancer at *any* member. At the leader the exchange is one round trip; at a follower the contacted node forwards the operation to the leader and answers the commit itself, so the client observes one round trip plus a one-way forwarding leg. No leader-aware routing is required for correctness; a load balancer that learns the leader's identity trades a fraction of a round trip for the availability of random assignment. {TODO: measure the follower path's forwarding leg against leader-aware routing in the cloud campaign.}

{TODO: situate against the transport direction of the industry. QUIC and HTTP/3 [12], [13] move applications away from in-order, in-path queuing toward connectionless datagrams; fragmentation and reassembly are handled below the application. The replicated state here is small metadata, so messages stay small and contention-driven drops and resends are the cost to avoid, not head-of-line blocking.}

{TODO: large-payload pattern. Where an application's values are large — agent logs, whose user inputs are small and tool responses large — the client writes the bulk value to object storage and consensus runs over the returned locator. State the pattern and its durability hand-off.}

B. Applications

An application of strong consistency is the replication of small state by consensus: the configuration authority Lamport, Malkhi and Zhou describe [22], and Vertical Paxos's auxiliary configuration master [23], are the precedents. This design embeds such an authority in the application — for small metadata, with ordinary replication performed in memory.

a) Short-lived advisory locks and leader leases: A replicated state machine can order acquisition, renewal and expiry decisions over a small record containing an owner, generation and expiry policy. An expiry command for an old generation must not revoke a newer grant. If a client uses ownership to control an external resource, that resource may also need to reject stale generations. A real-time leader lease additionally needs explicit clock and timing assumptions.

b) A membership authority for another algorithm: An application whose bulk algorithm uses gossip or another weaker consistency model may still need one agreed version of cluster membership, ownership assignments or configuration changes. An embedded replicated state machine can supply that authority, while the bulk algorithm disseminates or consumes its decisions.

c) *The configuration as replicated state:* In the Paxos family the configuration itself is replicated consensus state: Lamport, Malkhi and Zhou define a reconfigurable state machine whose configuration is a state component changed only by an executed reconfiguration command [22], and Vertical Paxos instead commits each new configuration to an auxiliary configuration master, which determines the set of acceptors of every configuration [23]. This design keeps that commitment inside the application: reconfiguration batches commit in the same slot stream as client values.

d) *The CORFU precedent:* CORFU provides a concrete precedent for compact configuration metadata alongside a larger storage system [15]. Its reconfiguration is patterned after Vertical Paxos; Section 2.5 describes storage units acting as acceptors in a Paxos-like protocol. Section 2.6 estimates that adding 0.1% writes keeps projection metadata below 25 MB for a 1 TB cluster under an adversarial workload. The described implementation uses a shared network drive for agreed projections.

e) *External coordination services:* ZooKeeper and etcd are established ways to supply coordination separately from an application. ZooKeeper offers linearizable state changes and per-client ordering, while local reads have weaker semantics [16]. etcd documents strict serializability for its key-value operations and distinguishes default linearizable reads from optional serializable reads [17]. An application must choose the operation semantics it actually needs.

A separately operated service also brings deployment and resource costs. For scale, etcd’s version 3.6 hardware guide gives a small-cluster example with dedicated two-vCPU, 8 GB AWS machines [18]. etcd can itself be embedded in a Go application [19]. The proposed combination here is small application-owned state, volatile normal-path replication, and replacement through fresh identities and overlapping quorums.

C. The host’s obligations

Discoverability, addressability, identity, and durability are first-class obligations of the host. Each is stated over happens-before relations of the host’s input and output to the network, to the boot fence, and to the durable state: no protocol message leaves before the boot fence records that the process runs, and no announcement names a new identity before that identity is flushed. A cold stop of all replicas followed by a cold start resumes in the same era, because the era is durable in the marker. These relations rest on the behaviour of the storage layer: a single flushed flag is not reliable enough on real file systems [7], [8], which is why the marker is a quorum of copies rather than a flag.

D. Scope

a) *Implementation correspondence:* The theorems assume protocol-history invariants. Their composition with the era-changing operational model is left as an exercise to the reader. The addendum [24] makes that exercise verifiable by stating its mapping, base case and preservation obligations, including clean-stop classification, identity fencing and state transfer before voting.

b) *Scope and availability:* Tables I and II replace one voter among three and five respectively. The addendum constructs paths between arbitrary weighted-majority endpoints with available majorities. Agreement follows from the history contract; progress additionally uses available recipients and a stable leader. Fresh identities permit repeated replacement while each committed boundary preserves the invariant.

c) *Meaning of strong consistency:* The mechanised result is per-slot agreement. A complete service-level linearizability argument also needs ordered execution, appropriate read semantics and a mapping between client operations and log entries. Lease guarantees require the additional conditions stated above.

IX. THE DEMONSTRATION: LUNET-LOCKS

The demonstration is `lunet-locks`, a runnable, fixed-membership advisory-lock service. It orders lock requests with `vrr-core`, the sans-IO Viewstamped Replication Revisited core that accompanies this formalisation: one total function maps a tick, an input message, and a state to a new state and the messages to send, and the host owns time, transport, storage, packetisation, threading, and naming. Clients speak newline-delimited JSON over TCP; replicas exchange the core’s binary datagrams over raw UDP; the service process is LuaJIT and libuv (the Lunet runtime, version-pinned and hash-verified), and a small Rust adapter drives the core and owns the lock state machine.

A. Components

The adapter instantiates the concrete core replica over a segmented log with weighted-majority quorums, running volatile stability: protocol state lives in quorum memory, an empty journal is provisioned at boot, and the only bytes the service ever fsyncs locally are the recovery nonce file. The adapter retains the tick clock (a monotonic value clamped against wall-clock regression), exactly-once reply correlation keyed by the client’s message identifier, and guard checks that re-validate every replicated operation entry — in Prepare, DoViewChange, StartView, RecoveryResponse, and NewState messages — against the lock service before the entry reaches the core. The Teal/Lunet wrapper owns sockets, TCP framing, peer source validation, and forwarding; a UDP peer envelope carrying a membership fingerprint guards the fixed membership at the transport boundary, so differently configured clusters cannot merge. {TODO: upcall-at-any-node latency check. The wrapper forwards a client operation to the leader when the contacted node is a follower, and the contacted node answers the commit; measure the leader path against the follower path, and record the availability–latency trade of leader-aware load-balancer routing. The demonstration’s connectionless datagram transport makes the forwarding leg a single hop with no connection state.} An append-only lock-event journal, a REST/WebSocket feed, and a console are observability only: the journal never participates in replication or recovery, and a journal error disables journaling without affecting the service path.

B. The rejoin path implemented

Every boot enters fenced recovery stamped with a fresh durable nonce. The nonce file is updated atomically — write, fsync, rename, parent-directory sync — and the new value is used as the recovery input’s tick, so per-node monotonicity holds across the two clock sources. Recovery needs a live quorum: a restarted node rejoins from quorum memory, installing the latest fenced view’s history and re-applying its committed suffix. A rolling single-node restart therefore recovers from the surviving members, while a simultaneous full-cluster loss forfeits whatever the quorum held, and an operator re-bootstrapping a lost cluster must do so only after all outstanding leases can no longer be valid.

A node outside the cluster — a lost durable state or a fresh identity — rejoins through the gossip path of Section VI: the join gossip to every known node, the witness list acted on by the leader, and the streamed keep-up to the first fence. The wire carries the gossip as its own message type, and the explicit-message tests pin the whole exchange; the section’s safety argument is the contract those tests hold the implementation to.

Membership is fixed for one process lifetime, so the reincarnation eviction of Section II — the forced weight sequence that replaces the crashed identity with a new one — is not exercised by this service. What the demonstration implements is the classic recovery boundary whose amnesia exposure motivates reincarnation, hardened with durable-nonce fencing: the fresh nonce is exactly the freshness ingress that the reincarnation design’s stale-episode rule realises, and a restarting node can never complete recovery with a stale identity stamp. Converting the eviction sequence into this service is the next integration step.

C. The lock and lease API

The client protocol has three operations, one JSON object per line over TCP. Every request carries a message identifier, a stable client identifier, an increasing request number, and a lock identifier; retries reuse the exact envelope, and replication deduplicates by (client, request number), replaying the prior reply. `get` returns the live lease on a lock, or null. `set` grants or renews a lease — a lease identifier, a holder identifier, and an expiry — and a lease is live only when its expiry is later than the execution time; an expired candidate is rejected, an absent or expired incumbent is free, and a live incumbent may be renewed or replaced only by the same holder. `release` removes a live lease only when both holder and lease identifier match, and is idempotent for a missing or expired lease. Lease expiry is what makes the diskless design safe to operate: a crashed holder’s locks free themselves when the lease lapses, without any recovery path trusting the holder’s lost state.

D. Test and hardening status

The service is exercised from several directions: a pure-Teal test suite and an FFI-boundary test suite for the lock service and cluster configuration; a Cerulean format gate and Cyan type checks; the Rust adapter’s checks, tests, and release build,

which depend on the exact pinned `vrr-core` git revision fetched over HTTPS; a three-process smoke test covering acquire, read, contention, release, reacquisition, expiry takeover, and a one-replica restart with a live quorum; and a thirty-second, three-datacentre lease-failover simulation that kills the observed lease holder every three seconds and verifies takeover within five seconds, exiting nonzero on a conflicting holder. The same simulation runs against a stable Docker cluster. CI runs on Linux for regular changes and a full platform matrix with packaged releases for tagged versions.

It is a working implementation under active hardening and testing, not a finished demonstration of the reincarnation protocol.

X. EXPERIMENTAL RESULTS: DESIGNS, NOT MEASUREMENTS

The claim under test is diskless low latency: rejoin through reincarnation is faster than paying a forced disk flush at the reincarnation boundary. The decision rule for the campaign: if the measured rejoin latency of E1 is slower than the forced-flush baseline of E2 at the same sample count, the diskless claim collapses — if rejoining is slower than doing disk flushes, nothing has been achieved. The system under test for all four designs is the `lunet-locks` demonstration of Section IX. Every number below is a placeholder of the form X.XX, showing the significant figures and decimal places the run will report — latencies in milliseconds with two decimals for the local rig and one decimal at tens of milliseconds, counts as integers — to be filled at run time from the lab book, one figure per lab-book row.

A. E1: node-kill rejoin latency

The design kills one voting node that is not the leader, rotated across all non-leader voting nodes of the three-datacentre `lunet-locks` cluster, for $k = \text{XXX}$ kill/rejoin cycles, with continuous competing lock traffic at the lease cadence throughout. Each iteration begins only after the previous rejoin has completed serving and a soak interval has elapsed, so no measurement is contaminated by the previous one. For each iteration the harness waits for steady state, kills the node’s process — Crash-Stop: volatile state lost, the node is by construction a different node on reopen — and the iteration ends when the reincarnated node has rejoined as a weight-0 standby, been walked back to voting weight by the leader’s forced sequence, and is again a voting, serving replica. The metric is the kill-to-rejoin-serving latency, one sample per iteration: p50 X.XX ms, p90 X.XX ms, p99 XX.X ms, max XX.X ms, mean X.XX ms, and the count of eras and view changes consumed per rejoin, X. Fig. 5 shows the designed chart: kill iteration on the horizontal axis, rejoin-serving latency on the vertical axis, one series per milestone of the forced sequence (standby rejoin at weight 0, voting again at weight 1). No data points exist yet; the chart is watermarked “dummy”.

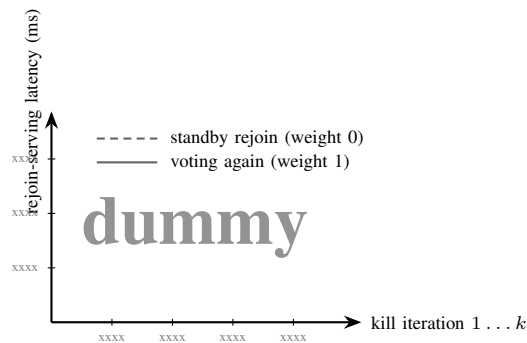


Fig. 5. E1: node-kill rejoin latency.

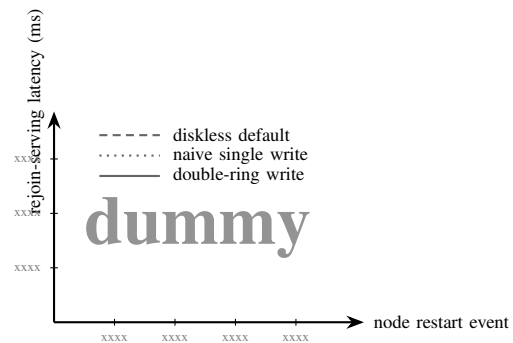


Fig. 6. E2: disk modes at the reincarnation boundary.

B. E2: disk modes at the reincarnation boundary

What does the diskless default save, in latency, against paying a forced flush at the reincarnation boundary? The design builds custom `lunet-locks` binaries, identical in every respect other than the durability behaviour at the reincarnation boundary — the point at which a reincarnating node durably records state before rejoining serving:

- *Diskless (the default)*. No durable writes at the reincarnation boundary; quorum memory is the only protocol state, and the recovery nonce file remains as shipped.
- *Naive single write (the latency baseline)*. One 4 KiB block write followed by `fsync` at the reincarnation boundary. Per the FAST’18 finding [4], stock consensus systems lose committed data under a single storage fault despite `fsync`; the variant is the latency baseline only, never a safety claim.
- *Double-ring write*. A TigerBeetle-style double write at the reincarnation boundary: one 4 KiB block of data plus a checksum header to the first WAL ring, then the header and checksum with no payload to the second WAL ring, the two rings spaced apart in separate 4 KiB-sector-aligned zones so the two checksum copies sit in different erasure blocks. Both flush variants carry fake data only, with no read-back: they measure write-and-flush latency and never reconstruct state from what was written.

Each variant runs E1’s procedure exactly, $k = \text{XXX}$ iterations per variant, cold counts. The metrics are the kill-to-rejoin-serving distribution per variant (percentiles as in E1) and, for the flush variants, the reincarnation-boundary flush latency itself per iteration. The planned report is the rejoin-latency delta: diskless median X.XX ms, naive-flush median XX.X ms, double-ring median XX.X ms, with tail figures XX.X ms at the ninth decile. The delta is the utility test: if reincarnation is slower than the forced flush, the diskless design has achieved nothing. Fig. 6 shows the designed chart: node restart event on the horizontal axis, rejoin-serving latency on the vertical axis, one series per build (diskless default, naive single write, double-ring write). No data points exist yet; the chart is watermarked “dummy”.

C. E3: Maelstrom brute force at three and five nodes

Does reincarnation hold consistency under brute-force fault injection? Jepsen’s Maelstrom drives the cluster under its fault

profile with reincarnation as the failure model: node crashes with volatile-state loss — the crashed node reincarnates rather than crash-recovers — plus partitions, with a Maelstrom adapter fronting the lock service as a compare-and-set register and competing lock clients at the lease cadence. The design runs the fault profile at three nodes, then at five, XXX runs per cluster size. Three checks decide each run: Maelstrom’s linearizability check passes over the register — no committed operation is lost, and no node rejoins under its old identity to overwrite a newer decision; every killed node reincarnates and rejoins serving, with no run ending in a permanently lost node; and no read ever reports two holders, expiry being judged only in the leader’s clock. All figures X.XX (medians, milliseconds) and XXX (run counts). A passing campaign supports the case that reincarnation eliminates the diskless-recovery weakness [3]: consistency under crash with successful rejoin, by construction rather than by recovery.

D. E4: cloud repetition

E1 and E2 are repeated on commodity cloud virtual machines, the six-node cluster laid out over three regions, two machines each — the correlated-infrastructure setting that weighted configurations address. Variables, procedure, metrics, and sample counts are E1’s and E2’s unchanged; only the environment differs. The planned report records XXX kills per configuration and zone-crossing medians of XX.X ms (E1) and XX.X ms (E2), with XX availability incidents. Fig. 7 shows the designed chart: experiment on the horizontal axis (E1 rejoin, E2 flush delta), latency on the vertical axis, one series per deployment (local cluster, cloud machines). No data points exist yet; the chart is watermarked “dummy”.

E. E5: the failover budget — N round trips, no disk flush

The design claim is that view failover costs N network round trips and *no* forced disk flush, where the classic diskful alternatives pay at least one flush on the fence path. The budget is measured, not assumed: the view change is decomposed into its wire segments and each segment timed. For the three-node cluster the segments are: suspicion (the phi-accrued suspect interval, floored by the host’s election knob), `STARTVIEWCHANGE` dissemination (1 RTT), `DOVIEWCHANGE` collection at the next view’s primary

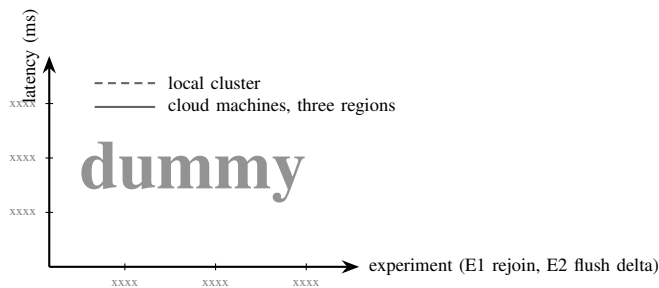


Fig. 7. E4: cloud repetition of E1 and E2.

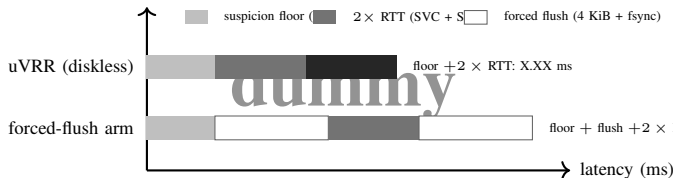


Fig. 8. E5: the failover budget. Stacked segments per arm; the forced-flush arm prices the classic diskful barrier on identical hardware in the same run, via the custom test build’s out-of-algorithm flush.

(pipelined with dissemination: 0 extra RTT when the messages cross), and STARTVIEW installation (1 RTT): the designed expectation is failover = suspicion floor + 2 RTT + 0 flushes. The counterfactual arm pays the same suspicion floor and round trips *plus* a forced flush before fencing (the VSR-1988-class stable-storage barrier) and a further flush before the new primary may serve installed state: suspicion floor + flush + 2 RTT + flush. Both arms are measured on the same hardware, the same run. The counterfactual is built by the forced-write rig of the campaign plan: a custom test build of the demonstration performs one 4 KiB write and `fsync` *outside* the algorithm — before the fenced node emits its view-change traffic and again before the new primary answers STARTVIEW — so the rig prices the diskful barrier without any uVRR code being made unsafe or rewritten to resemble the compared systems. The reported numbers are the per-segment medians and the totals: suspicion floor X.XX ms; per-RTT X.XX ms; total failover, ours, X.XX ms; total failover, forced-flush arm, XX.X ms; flush cost per boundary X.XX ms, expressed also in units of the same-run RTT, X.X RTTs. The last ratio is the hardware-agnostic headline: it should survive moving between loopback, local virtual machines, and cloud machines, where the absolute latencies will not. Fig. 8 shows the designed stacked budget chart, one bar per arm; no data points exist yet; the chart is watermarked “dummy”.

F. E6: suspicion calibration under jitter — what *phi* actually saw

The campaign owes an explanation of the detection floor, because the first rig runs showed the *phi* accumulator firing early: its estimate sat far below the true round-trip time, and under reconfiguration jitter its variance exploded. The runtime is LuaJIT plus FFI driving a Rust core with small messages, so the noise budget is genuinely small; the open question is whether the residual variance is garbage collection, scheduler

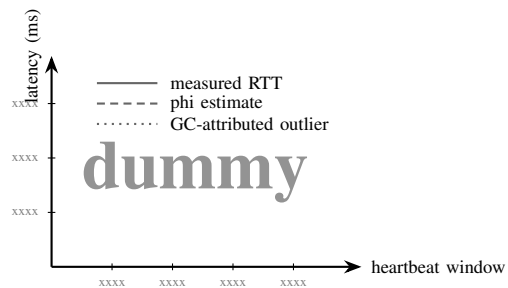


Fig. 9. E6: *phi* estimate against measured RTT, GC-attributed outliers marked.

jitter, or the estimator’s window shape, and E6 decides it by attribution rather than by guess. The rig samples, per heartbeat, the measured RTT, the *phi* estimate, the estimator window’s mean and variance, the LuaJIT allocation counter (`collectgarbage("count")`) before and after each window, and the Rust-side tick gap distribution; reconfiguration epochs are marked on the same axis. The designed report is the *phi* estimate against the measured RTT distribution with GC-attributed pauses marked: median estimate X.XX ms against median RTT X.XX ms, ninth-decile estimate XX.X ms against ninth-decile RTT XX.X ms, GC-attributed share of outlier windows XX.X percent, scheduler-attributed share XX.X percent, unexplained share XX.X percent. The decision rule: if the GC share explains the outlier windows, the estimator gets a variance-capped window and a stated floor; if scheduler jitter dominates, the floor moves to the measured jitter percentile and the paper says which. Fig. 9 shows the designed chart; no data points exist yet; the chart is watermarked “dummy”.

G. Cross-checks: the figures must be self-consistent

Every placeholder above carries an implicit order of magnitude, and the campaign checks the consistency before any number is quoted. The rules the lab book enforces: no loopback-harness figure may be quoted beside a cloud claim (loopback round trips are tens of microseconds; intra-zone cloud round trips are not); every failover figure must be at least the suspicion floor plus two measured round trips of the same rig — a failover quoted an order of magnitude faster than that rig’s RTT is a measurement error, not a result; every flush figure must be at least the device class’s write latency — a flush quoted an order of magnitude below the block device’s measured `fsync` is the same class of error; and ratios may only be formed within one rig and one run. Table III lists the checks; the lab-book template carries one row per figure and no figure may enter the paper with its row blank.

XI. RELATED WORK

Viewstamped Replication introduced the organisation of replication around views and a primary [1]; VRR revisits that organisation and separates normal operation, view change, recovery and reconfiguration [2]. Michael *et al.* identify the failure of diskless recovery to preserve view-change commitments and construct virtual stable storage [3]. uVRR takes a different route: it makes a crash final for the old identity, then brings

figure	rig	lower bound	check
E1 rejoin p50	local	X.XX ms (2 RTT + floor)	$\geq$ bound
E1 rejoin p50	cloud	XX.X ms (2 RTT + floor)	$\geq$ bound
E2 flush delta	local	X.XX ms (device fsync)	$\geq$ bound
E2 flush delta	cloud	XX.X ms (device fsync)	$\geq$ bound
E5 failover total	local	X.XX ms	$=$ floor + 2 RTT
E5 failover total	cloud	XX.X ms	$=$ floor + 2 RTT
E6 phi median	local	X.XX ms (RTT median)	$\approx$ RTT

TABLE III

CONSISTENCY CHECKS: EACH FIGURE’S RIG, ITS PHYSICAL LOWER BOUND, AND THE ARITHMETIC IDENTITY IT MUST SATISFY. X.XX MARKS THE PLACEHOLDER SHOWING THE SIGNIFICANT FIGURES THE RUN WILL REPORT; NO FIGURE ENTERS THE PAPER WITHOUT ITS ROW COMPLETED IN THE LAB BOOK.

the process back as a new member through reconfiguration. The replacement schedule connects that lifecycle to agreement across configurations.

Turner’s *Unbounded Pipelining in Dynamically Reconfigurable Paxos Clusters* supplies the central quorum geometry, era-based agreement argument and casting-vote construction [9]. Withholding the leader’s own promise is Turner’s mechanism. The present work applies it to reincarnation, proves weighted configuration reachability and an abstract casting-vote completion, and proves agreement for a concrete replacement instantiation from the protocol’s history invariants. The application discussion places this design alongside coordination services and CORFU, using their stated semantics and resource context.

REFERENCES

- [1] B. M. Oki and B. H. Liskov, “Viewstamped replication: A new primary copy method to support highly-available distributed systems,” in *ACM PODC*, 1988, pp. 8–17.
- [2] B. Liskov and J. Cowling, “Viewstamped replication revisited,” MIT CSAIL, Tech. Rep. MIT-CSAIL-TR-2012-021, 2012. [Online]. Available: <https://hdl.handle.net/1721.1/71763>
- [3] E. Michael, D. R. K. Ports, N. Kr. Sharma, and A. Szekeres, “Providing stable storage for the diskless crash-recovery failure model,” University of Washington, Tech. Rep. UW-CSE-16-08-02, Aug. 25, 2016. [Online]. Available: <https://syslab.cs.washington.edu/papers/diskless-tr16.pdf>
- [4] R. Alagappan, A. Ganesan, E. Lee, A. Albarghouthi, V. Chidambaram, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Protocol-aware recovery for consensus-based storage,” in *USENIX FAST*, 2018. [Online]. Available: <https://www.usenix.org/system/files/conference/fast18/fast18-alagappan.pdf>
- [5] L. Lamport, “Paxos made simple,” *ACM SIGACT News*, vol. 32, no. 4, pp. 51–58, 2001.
- [6] E. Gafni and L. Lamport, “Disk Paxos,” *Distributed Computing*, vol. 16, no. 1, pp. 1–20, 2003. [Online]. Available: <https://lamport.azurewebsites.net/pubs/disk-paxos-disc.pdf>
- [7] T. S. Pillai, V. Chidambaram, R. Alagappan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “All file systems are not created equal: On the complexity of crafting crash consistent applications,” in *USENIX OSDI*, 2014. [Online]. Available: <https://www.usenix.org/system/files/conference/osdi14/osdi14-paper-pillai.pdf>
- [8] V. Chidambaram, T. S. Pillai, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “Optimistic crash consistency: Application-aware fully-extensible crash consistency,” in *ACM SOSP*, 2013, doi: 10.1145/2517349.2522726.
- [9] D. Turner, “Unbounded pipelining in dynamically reconfigurable Paxos clusters,” revision 1A9DBA37, Aug. 14, 2017. [Online]. Available: <https://github.com/DaveCTurner/paxos-membership>. The proof artifact includes the consulted PDF and its digest.
- [10] J. Li, E. Michael, N. K. Sharma, A. Szekeres, and D. R. K. Ports, “Just say NO to Paxos overhead: Replacing consensus with network ordering,” in *USENIX OSDI*, 2016, pp. 467–483. [Online]. Available: <https://www.usenix.org/system/files/conference/osdi16/osdi16-li.pdf>
- [11] S. J. Park and J. Ousterhout, “Exploiting commutativity for practical fast replication,” in *USENIX NSDI*, 2019, pp. 47–64. [Online]. Available: <https://www.usenix.org/system/files/nsdi19-park.pdf>
- [12] J. Iyengar and M. Thomson, “QUIC: A UDP-based multiplexed and secure transport,” RFC 9000, May 2021, doi: 10.17487/RFC9000.
- [13] M. Bishop, “HTTP/3,” RFC 9114, June 2022, doi: 10.17487/RFC9114.
- [14] S. Massey, “Viewstamped replication revisited,” Aug. 12, 2026. [Online]. Available: <https://simbo1905.wordpress.com/2026/08/12/viewstamped-replication-revisited/>
- [15] M. Balakrishnan, D. Malkhi, J. D. Davis, V. Prabhakaran, M. Wei, and T. Wobber, “CORFU: A Distributed Shared Log,” *ACM Trans. Comput. Syst.*, vol. 31, no. 4, Art. 10, Dec. 2013, doi: 10.1145/2535930.
- [16] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free coordination for Internet-scale systems,” in *USENIX ATC*, 2010. [Online]. Available: https://www.usenix.org/events/usenix10/tech/full_papers/Hunt.pdf
- [17] etcd authors, “etcd API guarantees,” documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: https://etcd.io/docs/v3.6/learning/api_guarantees/
- [18] etcd authors, “Hardware recommendations,” documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: <https://etcd.io/docs/v3.6/op-guide/hardware/>
- [19] etcd authors, “Embedding etcd in a Go application,” documentation v3.6, accessed Sep. 11, 2026. [Online]. Available: https://etcd.io/docs/v3.6/dev-guide/golang_embed_pkg/
- [20] simbo1905, “The network is faster than the disk,” 12 April 2024, accessed 7 September 2026. <https://simbo1905.wordpress.com/2024/04/12/the-network-is-faster-than-the-disk/>
- [21] simbo1905, “Paxos Voting Weights,” 16 March 2017, accessed 7 September 2026. <https://simbo1905.wordpress.com/2017/03/16/paxos-voting-weights/>
- [22] L. Lamport, D. Malkhi, and L. Zhou, “Reconfiguring a state machine,” *ACM SIGACT News*, vol. 41, no. 1, pp. 63–73, 2010, doi: 10.1145/1753171.1753191.
- [23] L. Lamport, D. Malkhi, and L. Zhou, “Vertical Paxos and primary-backup replication,” in *Proc. 28th ACM Symp. Principles of Distributed Computing (PODC)*, 2009, pp. 312–313, doi: 10.1145/1582716.1582783.
- [24] S. Massey, “uVRR code and proof addendum,” 2026. [Online]. Available: <https://github.com/luu-lunet/uvrr-core>
- [25] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in *Automated Deduction – CADE 28*, 2021, pp. 625–635, doi: 10.1007/978-3-030-79876-5_37.
- [26] E. W. Dijkstra, “Why correctness must be a mathematical concern,” inaugural lecture, Nov. 18, 1979. [Online]. Available: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD07xx/EWD720.html>
- [27] E. W. Dijkstra, “Courtesy Dr. Birgit Schieder,” EWD1215, Austin, Sep. 17, 1995. [Online]. Available: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD12xx/EWD1215.html>
- [28] C. A. R. Hoare, “An axiomatic basis for computer programming,” *Communications of the ACM*, vol. 12, no. 10, pp. 576–580, 1969, doi: 10.1145/363235.363237.